



INSTITUTO SUPERIOR TÉCNICO

Departamento de Engenharia Informática

Management and Administration of It Infrastructures and Services

MEIC-A 2024-2025 – 1st Period

Group 19:

José Pereira - ist1103252

Diogo Rodrigues - ist1102848

Simão Sanguinho - ist1102082

1. Introduction

This project introduces BuyCrypto, a demo application crafted specifically for educational purposes. BuyCrypto allows users to simulate purchases of various cryptocurrencies—including Bitcoin, Ethereum, Solana, and Monero—offering a hands-on way to understand market dynamics and real-time simulated price changes.

A key feature of BuyCrypto is its user-friendly interface, which presents four available cryptocurrencies for purchase. Each transaction triggers a toast notification in the upper right corner to confirm a successful purchase. To mimic real-world price fluctuations, the application incorporates a random number generator within a specified range, creating a dynamic price update for each cryptocurrency.

A video showcasing the features of BuyCrypto, along with further technical details discussed later in this document, can be accessed at the following link: [\[insert link here\]](#).

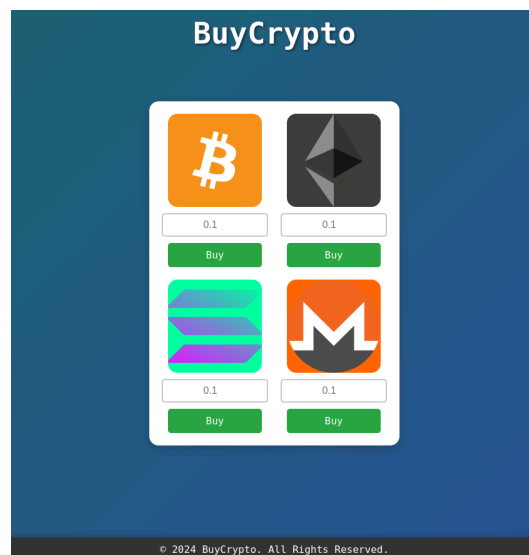


Figure 1: BuyCrypto interface

2. System Design and Architecture

Our project leverages a microservices architecture deployed on Google Cloud Platform (GCP) using Google Kubernetes Engine (GKE), a managed Kubernetes service that supports application orchestration, scalability, and automated recovery. To implement this architecture, we separated our application into four backend

microservices and a frontend interface that interacts with them. Each backend microservice is dedicated to handling simulated cryptocurrency purchases. These services connect to a Postgres database instance, which securely stores and manages transaction data.

To enhance system stability and performance, we integrated load balancers. This addition evenly distributes incoming requests across our microservices, which helps to prevent overload on any single service, improves response times, and ensures continuous availability.

For system monitoring and traceability, we integrated Grafana and Prometheus. These tools enable comprehensive tracking and visualization of key system metrics, supporting efficient oversight and performance analysis.

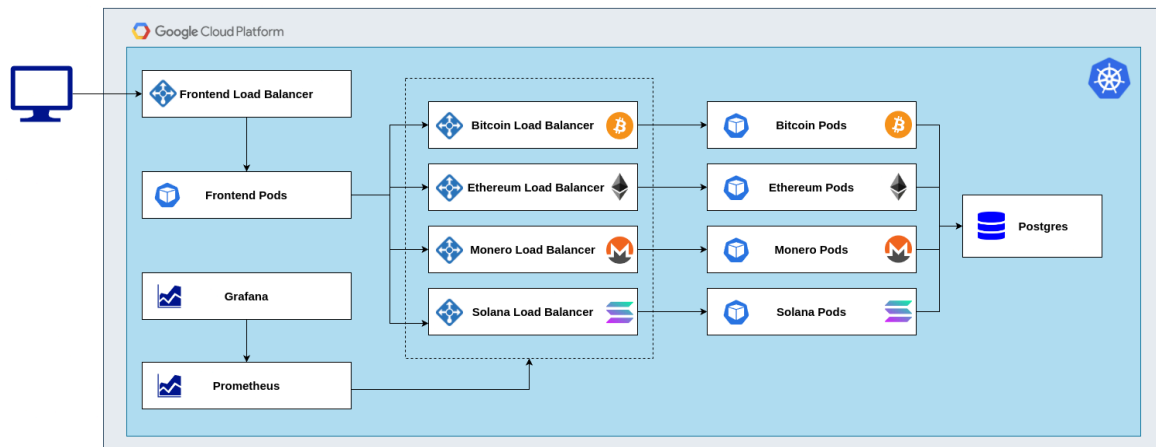


Figure 2: System Architecture

3. Infrastructure

3.1 System Deployment

Our infrastructure setup follows an Infrastructure as Code (IaC) methodology, utilizing Terraform to streamline the deployment and configuration of essential resources. To maintain a clear structure, we organized Terraform configurations into distinct modules, each dedicated to a specific application component. This modular design enhances ease of maintenance and enables efficient management of dependencies across different system elements, ensuring that each component can be individually adjusted without disrupting the overall infrastructure. The IaC approach thus provides a structured and automated foundation for our system's deployment and operational needs.

3.2 Frontend Service

To maintain simplicity and avoid complex dependency management, we built the frontend using only vanilla JavaScript, HTML, and CSS, with one exception: the use of *Toastify-js* for displaying transaction confirmation toasts. Relying solely on core web technologies allows us to avoid potential dependency conflicts and ensures that the application remains lightweight and easy to maintain. The choice to incorporate Toastify-js was made to efficiently implement visually appealing, responsive toasts for transaction confirmations, as building this functionality from scratch would be inefficient. This minimal approach not only enhances stability but also provides better control over the frontend's performance and keeps it free from the ongoing maintenance demands of larger frameworks.

3.3 Cryptocurrencies Services

The microservices for the various cryptocurrencies—Bitcoin, Ethereum, Solana, and Monero—are designed with a uniform architecture. Each service is built using Node.js and utilizes the Express framework for handling HTTP

requests and routing. For database interactions, we employ the `pg` package, which facilitates seamless communication with our database.

Each service exposes two endpoints: the root endpoint (`/`) accepts a quantity of the cryptocurrency to purchase, simulating a transaction while simultaneously saving the transaction record in the database. The second endpoint (`/metrics`) retrieves the total number of transactions for that specific cryptocurrency up to the current moment.

This consistent structure across all services not only simplifies development and maintenance but also allows for efficient updates and enhancements. Leveraging Node.js ensures that our services can handle multiple concurrent requests, providing a responsive experience for users simulating their cryptocurrency transactions.

3.4 Persistent Data

To manage our persistent data, we utilize an instance of PostgreSQL, ensuring robust and reliable storage of transaction information. This table includes essential columns: an ID column that uniquely identifies each transaction, a log column that captures the transaction details including the quantity purchased and the total price, and a timestamp column that records the exact time of the transaction.

For deployment, we used a Helm chart, which allowed us to define essential configuration details. We also included an initialization script to establish the database structure. However, we encountered a challenge: the script didn't accept the ``` character, which was needed to create multiple columns in a table. To work around this, we created the table structure initially, then added each column individually using separate ALTER commands.

3.5 Kubernetes Cluster

To effectively achieve automated deployment and scalability, we opted to deploy our application within a Kubernetes cluster on Google Kubernetes Engine (GKE). GKE offers a powerful managed Kubernetes service that simplifies the orchestration of containerized applications.

3.5.1 Containerization of Services

To begin the process of deploying our backend microservices and frontend to a GKE cluster, we first focused on containerizing each component. We utilized Docker to create the necessary images, which were then stored in a DockerHub repository. This approach allows us to replicate a production-like environment effortlessly, enabling a seamless transition to a development setup where all services operate as they would in a live setting.

3.5.2 Cluster creation

For hosting our application, we chose to deploy a regional cluster, which provides high availability by replicating its control panel across three different zones within the region of deployment. Similarly, the worker nodes are distributed across these various zones, ensuring that the cluster can remain operational even if one of the zones experiences a failure. This setup allows the cluster to continue serving requests through nodes that are provisioned in the remaining zones.

In our application's cluster, we utilized the default node pool that is automatically created during the cluster setup, initializing it with a standard configuration of one node per zone. Since regional clusters typically distribute nodes across three distinct zones, our setup begins with a node pool that includes three worker nodes, each situated in a different zone. This configuration enhances the resilience and availability of our application, allowing it to effectively handle potential disruptions.

3.5.3 Autoscaling

To deploy the service images we created earlier, we defined five Kubernetes services - one for the frontend and four for the backend microservices—along with the necessary deployments for pod creation. To optimize resource

usage while ensuring that our application can accommodate spikes in incoming traffic, we configured autoscaling within our cluster.

We implemented autoscaling in two key areas. First, we established horizontal pod autoscaling for all deployments associated with our application. These autoscalers monitor resource consumption across the pods serving each service and adjust the number of pod replicas accordingly. In our case, we focused on CPU utilization as the primary metric for scaling decisions. Specifically, we set the threshold at 50% of the maximum CPU capacity for the running pods. If utilization exceeds this limit, the service will scale its replicas within the predefined minimum and maximum limits specified during deployment.

Additionally, we enabled node auto-provisioning, which adjusts the number of active worker nodes based on current system requirements. This scaling strategy is closely linked to pod autoscaling, as the number of desired pods dictates how many nodes need to be added or removed. If a service becomes overloaded and requires additional pod replicas, the horizontal pod autoscaler will initiate this provisioning. If there are insufficient physical resources available on the current nodes to support the new pods, the cluster autoscaler will add more nodes as necessary. Conversely, if the load decreases, the system will automatically identify underutilized nodes and remove them to conserve resources.

3.6 Monitoring

To facilitate real-time performance monitoring of our system, we have integrated Prometheus and Grafana into our Kubernetes cluster. Prometheus is responsible for collecting and storing metrics, while Grafana serves as an intuitive dashboard for visualizing this data. Both tools have been deployed as services within the cluster.

We designed two distinct dashboards to display the metrics collected by Prometheus, as illustrated in Figure 2. The first dashboard focuses on the backend microservices, providing real-time insights into the number of transactions categorized by cryptocurrency. The second dashboard is dedicated to overall cluster performance, showcasing essential metrics such as memory usage, CPU load, disk storage consumption, and the current status of nodes, pods, and containers in real-time. Additionally, we implemented charts that monitor resource consumption for each service and display the number of replicas currently running for each application component. For deployment, we utilized Helm charts, which allowed us to specify the desired dashboards and configure the default admin account credentials (username: admin, password: Password123) directly in the values file.

3.7 Networking

To enable smooth communication between pods in our cluster, we set up a Virtual Private Cloud (VPC) network and deployed the cluster within it. This configuration allows internal services to interact through virtual IPs using Kubernetes DNS, keeping them shielded from external access. For services that require external access, like the Frontend, Grafana, and Prometheus, we configured the frontend service as a Load Balancer. This not only assigns a public IP address but also distributes incoming requests across all replicas to manage load effectively.

To further improve traffic management and security, we implemented a service mesh with Istio by adding sidecars to the application's pods. These sidecars intercept all traffic, enabling us to control and monitor both internal and external communications, which strengthens traffic handling and adds an extra layer of security.

3.8 Continuous Integration and Continuous Delivery

To streamline our development process, we implemented a robust CI/CD pipeline that automates both Continuous Integration (CI) and Continuous Delivery (CD) for our microservices. The CI component of the pipeline is

responsible for building new Docker images of the microservices and pushing them to Docker Hub. To optimize efficiency and reduce unnecessary rebuilds, we configured the pipeline to trigger image reconstruction only when changes are detected in the corresponding source code of a microservice.

In our pipeline, the deployment job is contingent upon a successful build job, ensuring that only those microservices with newly built images are eligible for deployment. For the CD aspect of our pipeline, we created a dedicated service account in Google Cloud Platform (GCP) that has the necessary permissions to access our Kubernetes cluster. This service account allows the cluster to authenticate within our GCP project, enabling it to execute kubectl commands seamlessly.

Since each of our services maintains a minimum number of replicas to ensure availability, we designed our Continuous Delivery (CD) strategy so that, when a new image is pushed to Docker Hub, our deployment process triggers the deletion of the existing pods for that specific microservice. As the pods are subsequently recreated, they automatically pull the latest version of the image, ensuring the application is running the most up-to-date features and fixes without manual intervention.

3.9 Evaluation

The stress test results highlight our application's strong performance and responsiveness under significant load. With a total of 100,000 requests sent at a concurrency level of 500, the system processed an average of 1,822 requests per second, with a minimal failure rate of 0. The application maintained a mean response time of 274 ms per request, with the vast majority (90%) of requests being completed within 213 ms or less. Even under high demand, the average transfer rate was around 4,108 KB/s, showcasing efficient data handling capabilities.

```
Concurrency Level:      500
Time taken for tests:   54.877 seconds
Complete requests:     100000
Failed requests:        0
Total transferred:      230900000 bytes
HTML transferred:      206000000 bytes
Requests per second:    1822.25 [#/sec] (mean)
Time per request:       274.385 [ms] (mean)
Time per request:       0.549 [ms] (mean, across all concurrent requests)
Transfer rate:          4108.97 [Kbytes/sec] received
```

Figure 3: Results for command `ab -n 100000 -c 500 -q -r http://<Frontend_IP>/`

As demonstrated in the test video, our application dynamically scaled in response to the increased request load. The replicas adjusted smoothly to the higher traffic, aided by the horizontal pod autoscaling configuration, ensuring continued service availability and responsiveness.

3.10 Conclusion

In developing this application, we aimed to build a scalable and reliable system that lets users easily simulate cryptocurrency transactions. By designing a microservices-based architecture deployed on Google Kubernetes Engine, we ensured the system can scale to meet demand while keeping resource usage efficient. Our CI/CD pipeline supports continuous improvement by automating build and deployment steps, while tools like Prometheus and Grafana help us monitor performance in real-time. This approach provides a resilient, user-friendly platform that supports a smooth and secure experience for all users.